

BUDGET BUDDY ENHANCEMENT

FULL-STACK TECHNICAL BLUEPRINT & BACKEND HANDOFF DOCUMENT

Target Platform: Node.js / Express • PostgreSQL / Prisma • Resend / Nodemailer • Gemini AI

Generated: September 2026 | Author: Antigravity AI | Version: 1.0.0 (Production Blueprint)

1. Executive Summary & Application Overview

Core functionality and verified client capabilities

Budget Buddy is an editorial-grade, modern personal finance telemetry platform and expense management system. It provides users with live financial telemetry, automated month-end financial reporting, net worth tracking, wealth allocation under the 50/30/20 budget framework, emergency vault management, and an AI-driven financial advisor assistant.

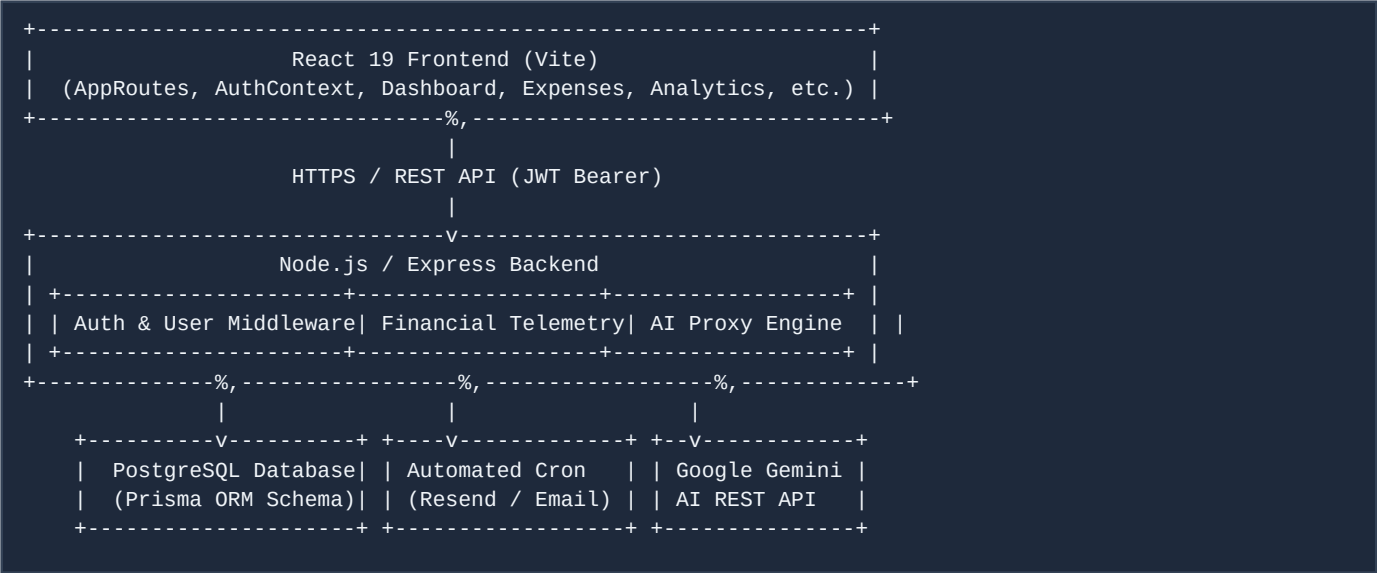
Key Verified Client-Side Capabilities:

Module / Feature	Description & Operation Scope
Financial Telemetry Engine	Computes Monthly Net Worth, Total Income, Total Expenses, Net Cashflow, Savings Rate, and Category Share.
50/30/20 Wealth System	Splits monthly income into Needs (50%), Wants (30%), and Savings/Vault (20%) reserves.
Transaction Ledger	Real-time ledger with search filtering, multi-category tags, custom dates, and deletion handlers.
Emergency Vault	Target vs Current emergency fund tracker with visual progress indicators.
Month-End Email Reports	Generates HTML & PDF financial statements with 6-month cashflow charts & AI audit insights.
AI Financial Advisor	Interactive AI assistant integrated with Google Gemini REST API (gemini-2.5-flash).
EMI & Virtual Card Studio	Virtual card designer & interactive monthly EMI calculation engine.
Verified Authentication	Dual-tab modal (GetStartedModal) and /auth route for sign up and strict user verification.

2. Technical Stack & System Architecture

Target end-to-end full-stack topology

The current application is built as a single-page React 19 application powered by Vite. Data is managed via a centralized localStorage service with custom window event dispatchers. The backend developer will replace localStorage calls with RESTful HTTPS endpoints backed by a Node.js Express server and PostgreSQL database.



3. Database Schema & Data Models (Entity-Relationship Blueprint)

PostgreSQL SQL DDL and Prisma ORM Model Specifications

3.1 Entity: users

```
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(255) NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  currency VARCHAR(50) DEFAULT 'INR (₹)',
  plan_tier VARCHAR(50) DEFAULT 'Pro Member',
  region VARCHAR(100) DEFAULT 'India (IN)',
  account_uid VARCHAR(100) UNIQUE NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

3.2 Entity: linked_accounts

```
CREATE TABLE linked_accounts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  bank_name VARCHAR(100) NOT NULL,
  account_type VARCHAR(100) NOT NULL,
  number_masked VARCHAR(20) NOT NULL,
  status VARCHAR(20) DEFAULT 'VERIFIED',
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

3.3 Entity: transactions

```
CREATE TABLE transactions (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,  
  title VARCHAR(255) NOT NULL,  
  category VARCHAR(100) NOT NULL,  
  amount NUMERIC(12, 2) NOT NULL,  
  type VARCHAR(20) CHECK (type IN ('income', 'expense')),  
  date DATE NOT NULL,  
  notes TEXT,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

3.4 Entity: budgets & budget_categories

```
CREATE TABLE budgets (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID UNIQUE REFERENCES users(id) ON DELETE CASCADE,  
  monthly_income NUMERIC(12, 2) DEFAULT 0,  
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE budget_categories (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  budget_id UUID REFERENCES budgets(id) ON DELETE CASCADE,  
  name VARCHAR(100) NOT NULL,  
  allocated_amount NUMERIC(12, 2) NOT NULL,  
  color VARCHAR(20) DEFAULT '#088fff'  
);
```

3.5 Entity: savings_goals & vault

```
CREATE TABLE savings_goals (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,  
  title VARCHAR(255) NOT NULL,  
  target_amount NUMERIC(12, 2) NOT NULL,  
  current_amount NUMERIC(12, 2) DEFAULT 0,  
  category VARCHAR(100) DEFAULT 'General',  
  deadline DATE,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE vault (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID UNIQUE REFERENCES users(id) ON DELETE CASCADE,  
  current_amount NUMERIC(12, 2) DEFAULT 0,  
  target_amount NUMERIC(12, 2) DEFAULT 200000,  
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

3.6 Entity: report_settings

```
CREATE TABLE report_settings (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID UNIQUE REFERENCES users(id) ON DELETE CASCADE,  
  enabled BOOLEAN DEFAULT true,  
  email VARCHAR(255) NOT NULL,  
  schedule VARCHAR(50) DEFAULT 'MONTH_END',  
  format VARCHAR(50) DEFAULT 'PDF_AND_HTML',  
  sections JSONB DEFAULT '{"cashFlowCharts": true, "categoryBreakdown": true, "rule503020": true, "advisor": true, "transactionLedger": true}',  
  last_sent_at TIMESTAMPTZ WITH TIME ZONE,  
  next_scheduled TIMESTAMPTZ WITH TIME ZONE  
);
```

4. RESTful API Endpoint Specifications

Complete router mapping for backend controllers

4.1 Authentication & Profile Router (/api/v1/auth & /api/v1/user)

Method	Endpoint Path	Access	Function	Payload
POST	/api/v1/auth/register	Public	Register new user account	{ name, email, password }
POST	/api/v1/auth/login	Public	User authentication & JWT issue	{ email, password }
POST	/api/v1/auth/logout	Protected	Invalidate active JWT session	None
GET	/api/v1/user/profile	Protected	Fetch active profile details	None
PUT	/api/v1/user/profile	Protected	Update user settings	{ name, currency, region }
GET	/api/v1/user/bank-accounts	Protected	List connected bank accounts	None
POST	/api/v1/user/bank-accounts	Protected	Link new financial account	{ bank, type, number }

4.2 Transaction Ledger Router (/api/v1/transactions)

Method	Endpoint Path	Access	Function	Payload
GET	/api/v1/transactions	Protected	Fetch transactions (with page, search, category filters)	Query Parameters
POST	/api/v1/transactions	Protected	Record new income or expense item	{ title, category, amount, type, date }
DELETE	/api/v1/transactions/:id	Protected	Delete transaction record by ID	URL Parameter :id
DELETE	/api/v1/transactions/clear-all	Protected	Purge all user transactions	None

4.3 Telemetry, Budget, Vault & Reports Router

Method	Endpoint Path	Access	Function	Payload
GET	/api/v1/telemetry/summary	Protected	Computes net worth, cashflow & 50/30/20 metrics	Returns JSON Telemetry
GET	/api/v1/telemetry/health	Protected	Computes Financial Health Index (0-100 score)	Returns Score & Pillar Breakdown
GET	/api/v1/budget	Protected	Fetch monthly income & category budget limits	None
PUT	/api/v1/budget	Protected	Update budget income and allocations	{ monthlyIncome, categories: [...] }

GET	/api/v1/savings/vault	Protected	Fetch emergency vault balance & target	None
PUT	/api/v1/savings/vault	Protected	Deposit / withdraw vault funds	{ current, target }
POST	/api/v1/reports/send-test	Protected	Dispatch instant month-end report email	{ email }
POST	/api/v1/ai/chat	Protected	Proxy prompt to Gemini API with context	{ prompt: string }

5. Core Telemetry Logic & Mathematical Reference

Server-side calculation specifications

The backend service must implement the following mathematical formulas to compute telemetry metrics dynamically:

A. Monthly Net Worth Formula:

Monthly Net Worth = (Total Monthly Income - Total Monthly Expenses) + Emergency Vault Reserves

B. 50/30/20 Wealth Allocation Engine:

Needs Amount (50%) = Monthly Income * 0.50 (Rent, Groceries, Utilities, EMIs)
Wants Amount (30%) = Monthly Income * 0.30 (Dining, Hobbies, Shopping)
Savings Amount (20%)= Monthly Income * 0.20 (Emergency Vault & SIP Investments)

C. Financial Health Index (0 - 100 Scale):

1. Savings Rate Metric (40% Weight) = (Net Savings / Total Income) * 100 [Benchmark: >= 20%]
2. Burn Rate Control (30% Weight) = Ratio of actual burn to 50/30 recommended budget
3. Vault Safety Margin (30% Weight) = Ratio of current vault balance to target reserve goal

6. Server-Side Automated Cron Job & Email Dispatch

Automated month-end financial statement delivery service

Create a server-side background worker (e.g. server/workers/reportCron.js) using node-cron and Resend or Nodemailer:

```
import cron from 'node-cron';
import { generateReportHTMLDocument } from '../services/reportGenerator.js';
import { sendEmailWithAttachment } from '../services/emailService.js';
import { prisma } from '../lib/prisma.js';

// Schedule to run at 23:50 on the 28th of every month
cron.schedule('50 23 28 * *', async () => {
  console.log('[CRON] Initiating Month-End Financial Report Dispatch...');

  const activeSubscriptions = await prisma.reportSettings.findMany({
    where: { enabled: true }
  });

  for (const config of activeSubscriptions) {
    try {
      const htmlContent = await generateReportHTMLDocument(config.userId);
      await sendEmailWithAttachment({
        to: config.email,
        subject: 'Official Budget Buddy Monthly Statement',
        html: htmlContent
      });

      await prisma.reportSettings.update({
        where: { id: config.id },
        data: { lastSent: new Date() }
      });
    } catch (err) {
      console.error(`[CRON ERROR] Failed for user ${config.userId}:`, err);
    }
  }
});
```

7. Frontend Integration Blueprint

Migrating from LocalStorage service to HTTP API client

Create an Axios client instance (src/services/apiClient.js) with JWT interceptors, then update userDataService.js to perform HTTP requests:

```
// client/src/services/apiClient.js
import axios from 'axios';

const apiClient = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL || 'http://localhost:5000/api/v1',
  headers: { 'Content-Type': 'application/json' }
});

apiClient.interceptors.request.use((config) => {
  const token = localStorage.getItem('budget_buddy_jwt_token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

export default apiClient;
```

8. Developer Setup & Environment Configuration

Environment secrets and CLI build commands

Frontend Environment (.env)

```
VITE_API_BASE_URL=http://localhost:5000/api/v1
VITE_GEMINI_API_KEY=your_google_gemini_api_key_here
```

Backend Environment (.env)

```
PORT=5000
NODE_ENV=development
DATABASE_URL=postgresql://postgres:password@localhost:5432/budget_buddy?schema=public
JWT_SECRET=super_secret_jwt_key_budget_buddy_2026
GEMINI_API_KEY=your_google_gemini_api_key_here
RESEND_API_KEY=re_123456789
SMTP_HOST=smtp.resend.com
SMTP_PORT=465
SMTP_USER=resend
SMTP_PASS=your_smtp_password
FRONTEND_URL=http://localhost:5173
```

9. Next Steps Summary for Future Developer

Action items for full project completion

Step	Implementation Task
1. Database Setup	Initialize PostgreSQL database and execute Prisma migrations using schema defined in Section 3.
2. Express REST API	Implement authentication controllers (JWT + bcrypt), transaction CRUD routes, and telemetry middleware.
3. AI Proxy Endpoint	Migrate direct Gemini API call from client GeminiAdvisorChat.jsx to server /api/v1/ai/chat proxy.
4. Month-End Cron Job	Configure Resend / Nodemailer transport in reportCron.js background worker for auto email dispatch.
5. Frontend Hookup	Replace LocalStorage helper methods in client/src/services/userDataService.js with apiClient requests.

